

Apache Airflow + Astronomer Astro

Module 2

Implementing Remote Execution Agents

Hands-On Implementation Lab Guide

Use Case	E-Commerce Order Analytics Pipeline
Lab Environment	AWS EC2 Ubuntu 22.04 + Minikube Kubernetes
Astronomer Astro	One active deployment (pre-configured)
Airflow Version	2.9+ (via Astro Runtime)
Skill Level	Intermediate (Freshers with EC2 access)
Estimated Duration	3 – 4 Hours

■ Learning Objectives

- Provision a Kubernetes cluster on your EC2 instance using Minikube
- Install and configure the Astronomer Hybrid Execution Agent (astro-agent)
- Connect the agent to your active Astro Cloud deployment
- Deploy and validate a real e-commerce analytics DAG using remote execution
- Monitor agent health, task logs, and worker pod lifecycle

TABLE OF CONTENTS

1.	Architecture Overview & Use Case	3
2.	Prerequisites & Environment Check	4
3.	Project Structure Setup	5
4.	Phase 1 – Setup Kubernetes Cluster (Minikube)	6
5.	Phase 2 – Install Execution Agents	9
6.	Phase 3 – Connect Agent to Astro Cloud	13
7.	Phase 4 – Deploy the Use-Case DAG	16
8.	Phase 5 – Validate Task Execution	21
9.	Troubleshooting & FAQs	25
1		
0.	Lab Cleanup	27

SECTION 1 – ARCHITECTURE OVERVIEW & USE CASE

1.1 Use Case: E-Commerce Order Analytics Pipeline

In this module you will build an end-to-end order analytics pipeline for a fictional e-commerce company **ShopStream**. The DAG simulates the daily batch workflow that:

- Extracts raw order data from a CSV source (mocked inside the DAG)
- Validates and cleans records (remove nulls, standardise currency)
- Aggregates revenue metrics by region and product category
- Writes a summary report to a local output directory
- Sends a Slack-style log notification on pipeline success

All heavy tasks run inside Kubernetes pods provisioned on your EC2 instance. The Astro Cloud control plane schedules the DAG; the **Remote Execution Agent** on your EC2 picks up work and submits pods to Minikube — giving you a true *hybrid* execution model without exposing your infrastructure to the public internet.

1.2 High-Level Architecture

Layer	Component	Runs On	Role
Control Plane	Astro Cloud UI + API	Astronomer SaaS	DAG scheduling, metadata, logs collection
Agent Layer	astro-agent (Helm chart)	EC2 / Minikube namespace	Polls Astro, spawns task pods
Execution Layer	Airflow Worker Pods	Minikube cluster (EC2)	Runs individual DAG tasks
Storage Layer	Persistent Volumes (hostPath)	EC2 disk	Shared DAG files and output artifacts

■ Agent Communication Flow

- Agent polls Astro Cloud API over HTTPS (outbound only — no inbound ports needed)
- Agent receives task run payload and submits a Kubernetes Job to Minikube
- Pod pulls the Airflow worker image, mounts DAG volume, executes the task
- Logs and XCom values are pushed back to Astro via the agent
- Astro UI reflects live task state; workers auto-terminate after task completion

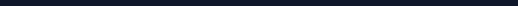
SECTION 2 – PREREQUISITES & ENVIRONMENT CHECK

2.1 What You Need Before Starting

Requirement	Detail	Check Command
EC2 Instance	t3.xlarge (4 vCPU, 16 GB RAM) — Ubuntu 22.04 LTS	
Storage	At least 30 GB free disk space on EC2	
Astro Account	Active Astronomer Cloud account (astro.io)	
Active Deployment	One deployment already running (noted in pre-lab setup)	
Astro CLI	v1.25+ installed on EC2 (astro version)	
Docker	Docker Engine 24+ (docker --version)	
kubectl	v1.28+ (kubectl version --client)	
Helm	v3.13+ (helm version)	
Python	3.11+ (python3 --version)	
Internet Access	EC2 can reach *.astronomer.run and registry.astronomer.io	

2.2 Quick Environment Validation

Run the following commands on your EC2 terminal to confirm all dependencies are in place before proceeding:

```
#  Verify all tools are present 
```

```
astro version
```

```
docker --version
```

```
kubectl version --client --short
```

```
helm version --short
```

```
python3 --version
```

```
curl -s https://astro.io/health | head -2
```

Expected output (versions may differ slightly):

```
Astro CLI, version 1.25.0
Docker version 24.0.7, build afdd53b
Client Version: v1.28.4
v3.13.1+g3547a4b
```

```
Python 3.11.6  
{"status": "healthy"}
```

■ Before You Proceed

- If astro version fails, re-run: `curl -sSL https://install.astronomer.io | sudo bash`
- If helm is missing: `sudo snap install helm --classic`
- If kubectl is missing: `sudo snap install kubectl --classic`
- Docker must be running: `sudo systemctl start docker && sudo usermod -aG docker $USER`
- Log out and back in after adding user to docker group

SECTION 3 – PROJECT STRUCTURE SETUP

3.1 Create Astro Project

We initialise a standard Astro project that holds our DAG, requirements, and Dockerfile. All commands are run from the EC2 terminal.

1 Create and initialise the project directory

```
# Create top-level workspace
mkdir -p ~/airflow-labs/module2-agents
cd ~/airflow-labs/module2-agents

# Initialise a new Astro project
astro dev init

# Confirm the scaffold was created
ls -la
```

```
.astro/
dags/
include/
plugins/
tests/
Dockerfile
packages.txt
requirements.txt
.env
```

2 Review default project files

The **astro dev init** command creates the following scaffold — understand each file's role:

File / Directory	Purpose
Dockerfile	Extends astro-runtime base; add system packages here
requirements.txt	Python package dependencies for your DAGs
packages.txt	OS-level apt packages installed at image build time
dags/	Place all your DAG .py files here
include/	Supporting files: SQL, configs, CSVs, helper modules
plugins/	Custom Airflow plugins (operators, hooks, sensors)
.env	Local environment variables (not pushed to Astro Cloud)

.astro/config.yaml	Project-level settings (deployment ID, workspace)
--------------------	---

3 Set Python dependencies

Edit `requirements.txt` to add the packages our DAG will use:

```
cat > requirements.txt << 'EOF'
apache-airflow-providers-cncf-kubernetes==8.3.0
apache-airflow-providers-http==4.10.0
pandas==2.2.1
pyarrow==15.0.2
EOF
```

4 Configure Dockerfile

Verify (or set) the Astro Runtime version in the Dockerfile:

```
# View current Dockerfile
cat Dockerfile

# If not already set, update to a supported runtime
# The FROM line should look like:
# FROM quay.io/astronomer/astro-runtime:11.3.0

# To update:
sed -i 's|FROM quay.io/astronomer/astro-runtime:.*|FROM
quay.io/astronomer/astro-runtime:11.3.0|' Dockerfile
cat Dockerfile
```

```
FROM quay.io/astronomer/astro-runtime:11.3.0
```

SECTION 4 – PHASE 1: SETUP KUBERNETES CLUSTER (MINIKUBE)

4.1 Why Minikube?

Minikube runs a single-node Kubernetes cluster inside a Docker container on your EC2 instance. It is the fastest way to get a production-equivalent Kubernetes environment for lab exercises without provisioning EKS or GKE. The Astronomer agent supports any CNCF-conformant cluster — Minikube, kind, EKS, GKE, AKS — so skills transfer directly.

4.2 Install Minikube

1 Download and install Minikube binary

```
# Download the latest stable Minikube release for Linux AMD64
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64

# Install to system path
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# Confirm installation
minikube version
```

```
minikube version: v1.32.0
commit: 8220a6eb4895...
```

4.3 Start the Minikube Cluster

2 Launch the cluster with adequate resources

The agent and its worker pods need sufficient CPU and memory. We allocate 4 CPUs and 8 GB RAM — adjust if your EC2 has fewer resources.

```
# Start Minikube – allocate resources appropriate for the EC2 instance
minikube start \
--driver=docker \
--cpus=4 \
--memory=8192 \
--disk-size=20g \
--kubernetes-version=v1.28.4 \
--addons=metrics-server \
--profile=astro-lab

# This takes 2-4 minutes on first run (pulls base images)
```

```
* [astro-lab] minikube v1.32.0 on Ubuntu 22.04
* Using the docker driver based on user configuration
```



```
* Starting control plane node astro-lab in cluster astro-lab
* Pulling base image v0.0.42 ...
* Creating docker container (CPUs=4, Memory=8192MB) ...
* Preparing Kubernetes v1.28.4 on Docker 24.0.7 ...
* Verifying Kubernetes components...
> metrics-server: Enabled
* Done! kubectl is now configured to use 'astro-lab' cluster
```

3 Verify cluster is healthy

```
# Check cluster status
minikube status --profile=astro-lab

# View nodes
kubectl get nodes -o wide

# View system pods
kubectl get pods -n kube-system
```

```
astro-lab
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

NAME STATUS ROLES AGE VERSION
astro-lab Ready control-plane 90s v1.28.4
```

4.4 Create Dedicated Namespace

4 Create the namespace for Airflow agent components

All agent resources (pods, services, secrets, ConfigMaps) live in the **airflow-agent** namespace — isolating them from other workloads.

```
# Create namespace
kubectl create namespace airflow-agent

# Label namespace for easier filtering
kubectl label namespace airflow-agent app.kubernetes.io/managed-by=astronomer

# Verify
kubectl get namespaces | grep airflow
```

```
airflow-agent Active 5s
```

4.5 Configure kubectl Context

5 Set Minikube as the active kubectl context

```
# Set context to astro-lab profile
kubectl config use-context astro-lab

# Verify current context
kubectl config current-context

# View full context details
kubectl config get-contexts
```

```
Switched to context 'astro-lab'.

CURRENT NAME CLUSTER AUTHINFO NAMESPACE
* astro-lab astro-lab astro-lab
```

4.6 Install Helm (if not present)

6 Verify or install Helm 3

```
# Check existing installation
helm version --short

# If not installed:
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# Add Astronomer Helm repository
helm repo add astronomer https://helm.astronomer.io
helm repo update

# Confirm repo added
helm repo list
```

```
v3.13.1+g3547a4b

NAME URL
astronomer https://helm.astronomer.io
```

■ Cluster Checkpoint

- minikube status shows all components as 'Running'
- kubectl get nodes shows STATUS = Ready
- Namespace airflow-agent exists and is Active
- helm repo list includes astronomer repo
- All checks green? Proceed to Phase 2.

SECTION 5 – PHASE 2: INSTALL EXECUTION AGENTS

5.1 Understanding the Astronomer Hybrid Agent

The **Astronomer Hybrid Execution Agent** (also called the *astro-agent* or *Hybrid Worker*) is a lightweight Kubernetes deployment that:

- Runs inside your cluster — entirely within your network perimeter
- Maintains an outbound HTTPS long-poll connection to Astro Cloud
- Receives serialised task payloads from the Astro scheduler
- Submits Kubernetes Jobs/Pods that execute the Airflow task code
- Streams logs and task state back to Astro Cloud via the same outbound channel

No inbound firewall rules or VPN are required. This makes it suitable for private VPC deployments and corporate environments with strict egress-only policies.

5.2 Retrieve Deployment Token from Astro UI

1 Log in to Astronomer Cloud and locate your deployment

You need the **Deployment API Token** to authorise the agent. Follow these steps in the Astro Cloud UI:

1	Navigate to cloud.astronomer.io and log in
2	Click Deployments in the left sidebar
3	Select your active deployment (the one pre-configured for this lab)
4	Go to the Details tab → scroll to API Keys section
5	Click + Create API Key → Name: lab-agent-key → Role: Deployment Admin
6	Copy the generated token immediately (shown only once)
7	Also note your Deployment ID from the URL bar (format: clXXXXXXXXXX)

2 Store credentials securely as environment variables on EC2

```
# Replace with your actual values
export ASTRO_DEPLOYMENT_ID="clXXXXXXXXXXXXXXXXXX"
export ASTRO_API_TOKEN="eyJhbGciOiJSUzI1NiIs..."
export ASTRO_WORKSPACE_ID="clYYYYYYYYYYYYYYYYYY"

# Persist to ~/.bashrc so they survive session restarts
echo "export ASTRO_DEPLOYMENT_ID=$ASTRO_DEPLOYMENT_ID" >> ~/.bashrc
echo "export ASTRO_API_TOKEN=$ASTRO_API_TOKEN" >> ~/.bashrc
echo "export ASTRO_WORKSPACE_ID=$ASTRO_WORKSPACE_ID" >> ~/.bashrc
source ~/.bashrc
```



```
# Reference the secret created in step 3
existingSecret: astro-agent-credentials

resources:
  requests:
    cpu: '250m'
    memory: '256Mi'
  limits:
    cpu: '500m'
    memory: '512Mi'

# Poll interval in seconds (how often agent checks for new tasks)
pollInterval: 5

worker:
  # Kubernetes namespace where task pods will be launched
  namespace: airflow-agent

  image:
    repository: quay.io/astronomer/astro-runtime
    tag: '11.3.0'
    pullPolicy: IfNotPresent

  resources:
    requests:
      cpu: '500m'
      memory: '1Gi'
    limits:
      cpu: '2000m'
      memory: '4Gi'

  # Max concurrent task pods per agent
  maxParallelism: 10

  # Volume for shared DAG files
  extraVolumes:
    - name: dags-volume
  hostPath:
    path: /home/ubuntu/airflow-labs/module2-agents/dags
    type: Directory

  extraVolumeMounts:
    - name: dags-volume
  mountPath: /usr/local/airflow/dags
  readOnly: true
```

```
serviceAccount:
create: true
name: astro-agent-sa

rbac:
create: true

EOF
```

5.5 Install the Agent via Helm

5 Run the Helm install command

```
# Install the Astronomer hybrid agent chart
helm install astro-agent astronomer/astro-agent \
--namespace airflow-agent \
--values ~/airflow-labs/module2-agents/agent-values.yaml \
--version 0.7.0 \
--wait \
--timeout 5m
```

```
NAME: astro-agent
LAST DEPLOYED: Tue May 13 10:32:17 2025
NAMESPACE: airflow-agent
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
Astronomer Hybrid Execution Agent installed successfully.
The agent is connecting to Astro Cloud...
```

6 Verify agent pods are running

```
# List all pods in the agent namespace
kubectl get pods -n airflow-agent

# Watch in real-time until Running
kubectl get pods -n airflow-agent -w
```

```
NAME READY STATUS RESTARTS AGE
astro-agent-6d8f9b7c4d-xk9pq 1/1 Running 0 45s
```

■ Common Installation Issues

- ImagePullBackOff → EC2 cannot reach quay.io; check security group outbound rules
- CrashLoopBackOff → Usually a malformed secret or wrong Deployment ID; re-check values
- Pending pod → Insufficient CPU/memory; run: `kubectl describe pod -n airflow-agent`
- Secret not found → Ensure namespace in `--namespace` matches secret's namespace

SECTION 6 – PHASE 3: CONNECT AGENT TO ASTRO CLOUD

6.1 Verify Agent Registration in Astro UI

Once the agent pod is running and credentials are correct, it automatically registers itself with your Astro Cloud deployment. This typically takes 30–60 seconds. Verify in both the Astro UI and via the agent logs.

1 Check agent logs for successful connection

```
# Get the agent pod name

AGENT_POD=$(kubectl get pod -n airflow-agent -l app.kubernetes.io/name=astro-agent -o
jsonpath='{.items[0].metadata.name}')

echo $AGENT_POD

# Stream agent logs

kubectl logs $AGENT_POD -n airflow-agent -f
```

```
[2025-05-13 10:32:30] INFO Starting Astronomer Hybrid Agent v0.7.0
[2025-05-13 10:32:31] INFO Connecting to https://astronomer.run/v1/agent
[2025-05-13 10:32:32] INFO Authenticating with deployment cl3ab9x7k0000...
[2025-05-13 10:32:33] INFO ✓ Authentication successful
[2025-05-13 10:32:33] INFO Registering agent with cluster: astro-lab (minikube)
[2025-05-13 10:32:34] INFO ✓ Agent registered. Agent ID: agt-4f7d2c9a
[2025-05-13 10:32:34] INFO Polling for tasks every 5s...
```

2 Verify in Astro Cloud UI

Navigate to the Astro Cloud UI and confirm the agent appears:

1	Go to cloud.astronomer.io → Deployments → select your deployment
2	Click the Workers tab (or Agents depending on Astro version)
3	You should see an entry: agt-4f7d2c9a with status CONNECTED
4	The Agent Version and Cluster name (astro-lab) should be displayed
5	The Last Heartbeat timestamp updates every 5–10 seconds

6.2 Configure Deployment to Use Hybrid Execution

3 Set the executor to KubernetesExecutor in Astro UI

For remote agent execution, the deployment must use the **Kubernetes Executor**. If your pre-configured deployment already uses it, skip to Step 4.

- Astro UI → Deployments → your deployment → Configuration tab
- Under Executor, select **Kubernetes Executor** (or Celery+Kubernetes if available)

- Under Execution Mode, select **Hybrid**
- In the Agent dropdown that appears, select your registered agent (agt-4f7d2c9a)
- Click Save → deployment will restart (takes ~2 minutes)

4 Push your Astro project to the deployment

Deploy the project code (including requirements and DAGs) to your Astro Cloud deployment. The agent will pull the image when executing tasks.

```
cd ~/airflow-labs/module2-agents

# Authenticate CLI with your Astro Cloud account
astro login astronomer.run

# Deploy to your active deployment
astro deploy --deployment-id $ASTRO_DEPLOYMENT_ID

# When prompted:
# Build and push? → y
# Image tag → press Enter (auto-generated)
```

```
Authenticated to Astronomer.
Building Docker image...
Successfully built 7a3f9c2d1e4b
Pushing image to registry.astronomer.io...
Deploy in progress for deployment cl3ab9x7k0000
✓ Deploy complete. Runtime version: 11.3.0
```

6.3 Configure Kubernetes Connection in Airflow

5 Add a Kubernetes connection in Astro UI

Some DAG tasks need an explicit Airflow **kubernetes_default** connection pointing to the in-cluster API. Since the agent runs inside the cluster, use in-cluster config:

- Astro UI → Deployments → your deployment → Connections tab
- Click + Add Connection
- Connection ID: **kubernetes_default**
- Connection Type: **Kubernetes**
- In Extra field, enter: `{"in_cluster": true}`
- Click Save

■ Agent Connection Checkpoint

- kubectl logs shows 'Agent registered' and 'Polling for tasks' messages
- Astro UI Workers tab shows agent status as CONNECTED
- Deployment executor is set to Kubernetes + Hybrid mode
- astro deploy completed successfully (Runtime 11.3.0)
- kubernetes_default connection added in Airflow UI

SECTION 7 – PHASE 4: DEPLOY THE USE-CASE DAG

7.1 DAG Overview: ShopStream Order Analytics

The DAG **shopstream_order_analytics** is a daily batch pipeline with 5 tasks. Each task runs as an isolated Kubernetes pod, orchestrated by the agent:

Task ID	Operator	Description
generate_orders	PythonOperator	Generates 1000 synthetic orders (mocked extract)
validate_orders	PythonOperator	Drops nulls, validates schema, flags anomalies
aggregate_metrics	PythonOperator	Computes revenue by region and category using pandas
write_report	PythonOperator	Saves aggregated CSV to /tmp/shopstream/reports/
notify_success	BashOperator	Echoes pipeline summary to logs (mock notification)

7.2 Create the DAG File

1 Create the DAG Python file

```
mkdir -p ~/airflow-labs/module2-agents/dags
cat > ~/airflow-labs/module2-agents/dags/shopstream_order_analytics.py << 'DAGEOF'
```

Paste the complete DAG code below into that file. Each section is explained with comments:

7.2.1 Imports and Constants

```
from __future__ import annotations
import os, random, csv, logging
from datetime import datetime, timedelta
import pandas as pd
from airflow.decorators import dag, task
from airflow.operators.bash import BashOperator
from airflow.models.param import Param

# Constants
REGIONS = ['North', 'South', 'East', 'West', 'Central']
CATEGORIES = ['Electronics', 'Apparel', 'Home', 'Sports', 'Books']
OUTPUT_DIR = '/tmp/shopstream/reports'
NUM_ORDERS = 1000
```

7.2.2 DAG Definition and Default Args

```
default_args = {
    'owner' : 'data-engineering-team',
    'depends_on_past' : False,
    'retries' : 2,
    'retry_delay' : timedelta(minutes=3),
    'email_on_failure': False,
}

@dag(
    dag_id = 'shopstream_order_analytics',
    description = 'Daily ShopStream order analytics – Module 2 use case',
    schedule = '@daily',
    start_date = datetime(2025, 1, 1),
    catchup = False,
    default_args = default_args,
    tags = ['shopstream', 'analytics', 'module2', 'k8s-agent'],
    params = {
        'num_orders': Param(1000, type='integer', description='Orders to generate'),
    },
)

def shopstream_order_analytics():
```

7.2.3 Task 1 — Generate Orders

```
@task(task_id='generate_orders')
def generate_orders(**context) -> str:
    '''Generate synthetic order data and save to CSV.'''
    num = context['params']['num_orders']
    log = logging.getLogger(__name__)
    log.info(f'Generating {num} synthetic orders...')

    os.makedirs(OUTPUT_DIR, exist_ok=True)
    raw_path = f'{OUTPUT_DIR}/raw_orders.csv'

    with open(raw_path, 'w', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=[
            'order_id', 'customer_id', 'region',
            'category', 'amount', 'status'])
        writer.writeheader()
        for i in range(1, num + 1):
            writer.writerow({
```

```
'order_id' : f'ORD-{i:05d}',
'customer_id': f'CUST-{random.randint(1,200):04d}',
'region' : random.choice(REGIONS),
'category' : random.choice(CATEGORIES),
'amount' : round(random.uniform(5.0, 999.0), 2),
'status' : random.choice(['COMPLETED', 'PENDING', 'CANCELLED']),
})
log.info(f'Orders written to {raw_path}')
return raw_path
```

7.2.4 Task 2 — Validate Orders

```
@task(task_id='validate_orders')
def validate_orders(raw_path: str) -> str:
    '''Validate schema, drop nulls, filter CANCELLED orders.'''
    log = logging.getLogger(__name__)
    df = pd.read_csv(raw_path)
    log.info(f'Loaded {len(df)} rows for validation')

    # Drop rows with any null values
    before = len(df)
    df = df.dropna()
    log.info(f'Dropped {before - len(df)} null rows')

    # Keep only COMPLETED orders for revenue analytics
    df = df[df['status'] == 'COMPLETED']
    log.info(f'Retained {len(df)} COMPLETED orders')

    # Ensure amount is float
    df['amount'] = df['amount'].astype(float)

    clean_path = f'{OUTPUT_DIR}/clean_orders.csv'
    df.to_csv(clean_path, index=False)
    log.info(f'Clean data saved to {clean_path}')
    return clean_path
```

7.2.5 Task 3 — Aggregate Metrics

```
@task(task_id='aggregate_metrics')
def aggregate_metrics(clean_path: str) -> str:
    '''Aggregate revenue by region and category.'''
    log = logging.getLogger(__name__)
    df = pd.read_csv(clean_path)

    # Revenue by region
```

```

region_rev = (
df.groupby('region')['amount']
.agg(total_revenue='sum', order_count='count', avg_order='mean')
.round(2)
.reset_index()
)

# Revenue by category
cat_rev = (
df.groupby('category')['amount']
.agg(total_revenue='sum', order_count='count', avg_order='mean')
.round(2)
.reset_index()
)

log.info('Region revenue summary:')
log.info(region_rev.to_string())
log.info('Category revenue summary:')
log.info(cat_rev.to_string())

agg_path = f'{OUTPUT_DIR}/aggregated_metrics.csv'
combined = pd.concat([
region_rev.assign(dimension='region', dimension_value=region_rev['region']),
cat_rev.assign(dimension='category', dimension_value=cat_rev['category'])
], ignore_index=True)
combined.to_csv(agg_path, index=False)
return agg_path

```

7.2.6 Task 4 — Write Report

```

@task(task_id='write_report')
def write_report(agg_path: str) -> str:
    '''Generate a human-readable text report.'''
    log = logging.getLogger(__name__)
    df = pd.read_csv(agg_path)
    date = datetime.now().strftime('%Y-%m-%d')

    report_path = f'{OUTPUT_DIR}/daily_report_{date}.txt'
    with open(report_path, 'w') as f:
        f.write(f'ShopStream Daily Report – {date}\n')
        f.write('=' * 50 + '\n\n')
        for dim in ['region', 'category']:
            subset = df[df['dimension'] == dim]

```



```
astro deploy --deployment-id $ASTRO_DEPLOYMENT_ID --force
```

```
Building Docker image...
```

```
Successfully built 8b4e1c3f2a7d
```

```
Pushing image...
```

```
✓ Deploy complete. Your DAG will appear in the Airflow UI in ~1 minute.
```

■ DAG Deployment Checkpoint

- DAG file is present in dags/ directory on EC2
- astro deploy completed with exit code 0
- shopstream_order_analytics appears in Astro UI DAGs list
- DAG shows tags: shopstream, analytics, module2, k8s-agent
- No import errors shown in the DAG's detail page

SECTION 8 – PHASE 5: VALIDATE TASK EXECUTION

8.1 Trigger the DAG Manually

We trigger the DAG manually to validate the entire agent → Kubernetes → task execution chain before relying on the scheduled run.

1 Trigger via Astro UI

- In Astro UI, go to Deployments → your deployment → Open in Airflow
- In the Airflow UI, find **shopstream_order_analytics** in the DAGs list
- Click the **Trigger DAG** button (■ icon on the right)
- In the trigger dialog, leave num_orders as 1000 → click Trigger
- Click on the DAG name to open the Grid view and watch tasks run

2 Trigger via Astro CLI (alternative)

```
# Trigger DAG run via CLI
astro deployment airflow-variable list --deployment-id $ASTRO_DEPLOYMENT_ID

# OR use the Airflow REST API directly
# First get the deployment URL from Astro UI (Settings > General > Airflow URL)
AIRFLOW_URL="https://.astronomer.run"

curl -X POST \
  $AIRFLOW_URL/api/v1/dags/shopstream_order_analytics/dagRuns \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $ASTRO_API_TOKEN" \
  -d '{"conf": {"num_orders": 500}}'
```

8.2 Monitor Task Pod Lifecycle on Kubernetes

3 Watch pods being created and completed

As the DAG runs, each task spawns a dedicated Kubernetes pod in the **airflow-agent** namespace. Watch this in real-time:

```
# Watch pods appear and complete (refresh every 2s)
kubectl get pods -n airflow-agent -w

# In a second terminal – see all pods including completed
kubectl get pods -n airflow-agent --show-labels
```

```
NAME READY STATUS RESTARTS AGE
astro-agent-6d8f9b7c4d-xk9pq 1/1 Running 0 12m
airflow-shopstream-generate-orders-abc12 0/1 Pending 0 3s
```

```
airflow-shopstream-generate-orders-abc12 1/1 Running 0 8s
airflow-shopstream-generate-orders-abc12 0/1 Completed 0 24s
airflow-shopstream-validate-orders-def34 0/1 Pending 0 25s
airflow-shopstream-validate-orders-def34 1/1 Running 0 30s
airflow-shopstream-validate-orders-def34 0/1 Completed 0 45s
... (subsequent tasks follow the same pattern)
```

4 Inspect individual task pod logs

```
# Get the name of a completed pod
kubectl get pods -n airflow-agent --field-selector=status.phase=Succeeded

# View logs for a specific task pod
kubectl logs airflow-shopstream-generate-orders-abc12 -n airflow-agent

# Or use the Airflow UI log viewer:
# DAG Run → click task cell → Logs tab
```

```
[2025-05-13 10:45:12,034] {shopstream_order_analytics.py:48} INFO - Generating 1000 synthetic
orders...

[2025-05-13 10:45:12,089] {shopstream_order_analytics.py:64} INFO - Orders written to
/tmp/shopstream/reports/raw_orders.csv

[2025-05-13 10:45:12,090] {taskinstance.py:1831} INFO - Marking task as SUCCESS.
```

8.3 Verify DAG Run Completion in Airflow UI

5 Inspect the Grid View

The Airflow Grid view shows task execution state for each DAG run. All 5 tasks should show **success (green)** status:

generate_orders	✓ Success	~15s	Generated 1000 orders, CSV written
validate_orders	✓ Success	~10s	Retained ~650 COMPLETED orders
aggregate_metrics	✓ Success	~12s	Revenue computed by region & category
write_report	✓ Success	~8s	Report file created with date stamp
notify_success	✓ Success	~3s	Pipeline summary echoed to logs

8.4 Validate Output Files on EC2

6 Confirm output artifacts were created

Task pods write output files to /tmp/shopstream/reports/ inside the Minikube node. Check the output directory:

```
# SSH into the Minikube node
minikube ssh --profile=astro-lab
```

```
# Inside the Minikube node:
ls -lh /tmp/shopstream/reports/

# View the daily report
cat /tmp/shopstream/reports/daily_report_$(date +%Y-%m-%d).txt

# Exit Minikube SSH
exit
```

```
total 96K
-rw-r--r-- 1 root root 42K May 13 10:45 raw_orders.csv
-rw-r--r-- 1 root root 28K May 13 10:45 clean_orders.csv
-rw-r--r-- 1 root root 3.2K May 13 10:45 aggregated_metrics.csv
-rw-r--r-- 1 root root 1.1K May 13 10:45 daily_report_2025-05-13.txt

ShopStream Daily Report – 2025-05-13
=====

REGION BREAKDOWN
-----
North Revenue: $26,847.32 Orders: 134
South Revenue: $24,312.90 Orders: 121
East Revenue: $27,540.11 Orders: 138
West Revenue: $25,901.44 Orders: 129
Central Revenue: $23,198.67 Orders: 128
```

8.5 Monitor Agent Health Metrics

7 Check resource usage of agent and worker pods

```
# View resource usage (requires metrics-server – enabled earlier)
kubectl top pods -n airflow-agent

# View agent deployment details
kubectl describe deployment astro-agent -n airflow-agent

# View agent events (useful for debugging)
kubectl get events -n airflow-agent --sort-by='.lastTimestamp'
```

```
NAME CPU(cores) MEMORY(bytes)
astro-agent-6d8f9b7c4d-xk9pq 35m 148Mi
```

8 Verify agent status in Astro UI after successful run

- Astro UI → Deployments → your deployment → Workers tab
- Agent shows: Status CONNECTED, Last Heartbeat < 30s ago
- Tasks Completed Today: 5 (one run × 5 tasks)

- No error indicators or disconnection warnings

■ Final Validation Checklist — All 5 Tasks Must Show Green

- generate_orders: raw_orders.csv exists with 1000 rows
- validate_orders: clean_orders.csv exists with ~650 COMPLETED rows
- aggregate_metrics: aggregated_metrics.csv has region and category rows
- write_report: daily_report.txt exists and contains revenue data
- notify_success: Logs show pipeline complete message
- Agent pod is still Running after the DAG completes
- Astro UI shows all 5 tasks as SUCCESS in the Grid view

SECTION 9 – TROUBLESHOOTING & FAQs

9.1 Common Issues and Fixes

9.1 Agent pod in CrashLoopBackOff

Diagnostic command:

```
kubectl logs -n airflow-agent --previous
```

Solutions:

- Check that astro-agent-credentials secret has correct non-empty values
- Verify ASTRO_DEPLOYMENT_ID matches the deployment in Astro UI
- Confirm API token has Deployment Admin role, not Viewer

9.2 Task pods stuck in Pending

Diagnostic command:

```
kubectl describe pod -n airflow-agent
```

Solutions:

- Look for Insufficient CPU / Insufficient memory in Events section
- Reduce worker resource requests in agent-values.yaml (cpu: 200m, memory: 512Mi)
- Run: minikube status to confirm cluster is healthy
- Run: kubectl top nodes to check node capacity

9.3 DAG not appearing in Airflow UI

Diagnostic command:

```
astro deploy --deployment-id $ASTRO_DEPLOYMENT_ID
```

Solutions:

- Wait 2 minutes after deploy for sync; refresh the UI
- Check DAG file has no syntax errors: `python3 -c 'import ast; ast.parse(open("dags/shopstream_order_analytics.py").read())'`
- In Astro UI, check for Import Errors in the DAG detail page

9.4 Task logs not visible in Airflow UI

Diagnostic command:

```
kubectl logs -n airflow-agent
```

Solutions:

- Agent may have lost connection; check agent logs for reconnect messages
- Ensure the agent pod is still Running during task execution
- Check Astro UI network connectivity: `curl -s https://astronomer.run/health`

9.5 File not found errors in DAG tasks

Diagnostic command:

```
minikube ssh --profile=astro-lab -- ls /tmp/shopstream/
```

Solutions:

- The /tmp/shopstream directory must exist in the Minikube node
- Tasks create this directory with `os.makedirs` — confirm task ran without errors
- If using `hostPath` volumes, verify the path exists before the first run

9.2 Agent Upgrade

To upgrade the agent to a newer Helm chart version without downtime:

```
# Update repo cache
helm repo update astronomer

# Upgrade in-place (keeps existing secret and namespace)
helm upgrade astro-agent astronomer/astro-agent \
--namespace airflow-agent \
--values ~/airflow-labs/module2-agents/agent-values.yaml \
--version 0.8.0
```

SECTION 10 – LAB CLEANUP

10.1 Remove Lab Resources

After completing the lab, clean up all Kubernetes and local resources to free EC2 disk and memory.

1 Uninstall the Helm release

```
helm uninstall astro-agent --namespace airflow-agent  
  
release 'astro-agent' uninstalled
```

2 Delete the Kubernetes namespace

```
# Deletes all pods, secrets, services in the namespace  
kubectl delete namespace airflow-agent  
  
namespace 'airflow-agent' deleted
```

3 Stop and delete the Minikube cluster

```
# Stop the cluster (preserves disk state)  
minikube stop --profile=astro-lab  
  
# Permanently delete the cluster  
minikube delete --profile=astro-lab  
  
* Deleting 'astro-lab' in docker ...  
* Removed all traces of the 'astro-lab' cluster.
```

4 Revoke the Deployment API Key in Astro UI

- Astro UI → Deployments → your deployment → API Keys tab
- Find lab-agent-key → click the trash icon → confirm deletion
- This ensures the lab token cannot be used after the session

■ Module 2 Complete!

- You successfully provisioned a Minikube Kubernetes cluster on EC2
- Installed the Astronomer Hybrid Execution Agent via Helm
- Connected the agent to your active Astro Cloud deployment
- Deployed and validated the ShopStream order analytics DAG
- Each task ran as an isolated Kubernetes pod — hybrid cloud execution confirmed
- Proceed to Module 3: Advanced DAG Authoring with Dynamic Task Mapping